

Virtuális memória kezelés

Az előadás anyaga

Dr. Vadász Dénes: Operációs rendszerek
jegyzet és URL linkek alapján készült el

Témakörök

- Memóriamenedzselési osztályok, valós MM.
- Virtuális memória – általánosságba.
- Lapozó rendszer.
- Szegmentáló rendszer.
- NT, Linux memória kezelés.

Memóriamenedzsment osztályok - ismétlés

Memória menedzselés szerint osztályozás:

1. *valós címzésű rendszerek:*

- monoprogramozású,
- multiprogramozású rendszerek:
 - fix partíció,
 - változó partíciós rendszerek.

Memóriamenedzsment osztályok - ismételés

2. virtuális címzésű rendszerek:

- lapozós (paging) rendszerek,
- szegmens-leképző, ki-besöprő rendszerek,
- szegmentáló, ki-besöprő és ugyanakkor lapozó rendszerek.

Virtuális memória kezelés

„Miért nevezzük *virtuális memóriának*? Azért

Azért virtuális, mert ez ilyen formában nem létezik, ez a terület a valóságban egy háttértáron található és a lineáris címeket átalakítja olyan formájúvá, ahogy azt a diszkek igénylik (lemezoldal, sáv, szektor, blokk) – ezt az OS vezérli.

Virtuális memória kezelés

Virtuális memória kezelés - *Virtuális címezést* használ.

Tehát, *virtuális címek* vannak, ezekhez *fizikai rekesz* kell biztosítani a *memóriában* vagy a *háttértáron*.

A leképzéshez nyilvántartás kell.

A nyilvántartás méretének csökkentése érdekében a *virtuális címtartományt blokkokra* kell osztani és a *blokkokat egyben* kell elhelyezni, így elég csak a *blokk kezdetet megjegyezni*.

Virtuális memória kezelés

A blokkokra osztás fajtája szerint a virtuális memória kezelés lehet:

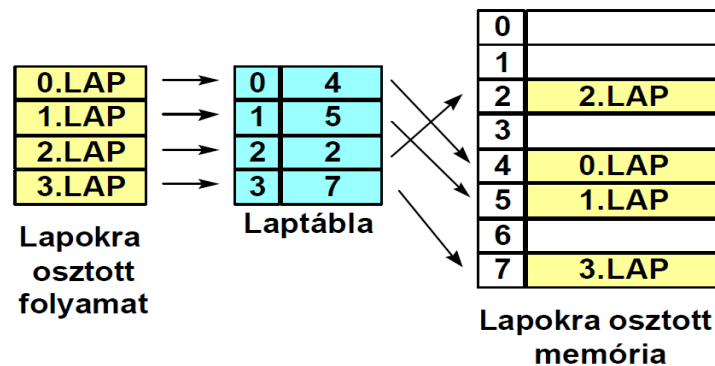
- Lapozásos,
- Szegmentálásos,
- Vegyes.”

Lapozás - általánosságban

„Osszuk fel a rendelkezésre álló *operatív memória területet egyforma és viszonylag kisméretű egységekre, ún. lapokra*.

Az ábrán az *operatív memória 8 lap méretű* és ebből:

- pillanatnyilag a 0., az 1., a 3. és a 6. lapot foglalt,
- míg a 2., a 4., az 5. és a 7. lap üres – sárga jelölt!



Forrás: Knapp G: OS

Lapozás - általánosságban

Az OS minden egyes *processz betöltésekor* létrehoz a processz számára egy *laptáblát*, amely a *logikai lapokhoz hozzárendeli a fizikai lapot*.

Összességében **4 üres lap van** (2,4, , 5, 7,) betölthetünk egy olyan processz, amely *4 lapot igényel*.

Mivel 4 lapot szeretnénk betölteni, ezért *4 soros laptáblát* kell létrehozni.

Lapozás - általánosságban

A laptábla első sora az hogy a 0. sorszámú logikai lap a 4. sorszámú fizikai lap helyére került az operatív memóriában - így tovább.

Kérdés az, hogy hogyan találja meg a memóriában az egyes utasításokat és adatokat.

Mert pl. a fordítóprogram szerint, ugorjon el a *program elejéhez képesti* 16. sorra – de hol van a 16.sor.

Lapozás - általánosságban

Tételezzük fel: *minden lap 10 sorból* áll, amelyeket 0 és 9 közötti számokkal látunk el (lap mérete 2 hatványai).

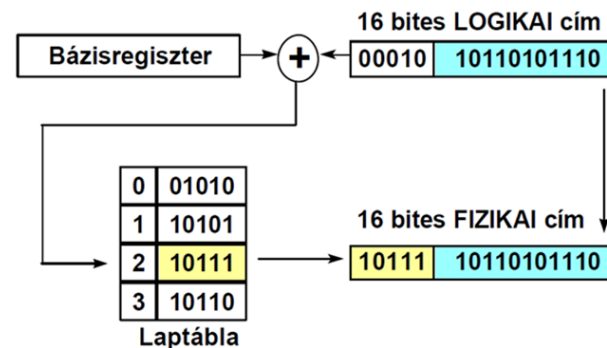
Számolás: a 16. logikai sor az *1-es számú logikai lapon* helyezkedik el és azon belül ez a *6. számú sor* - ezt úgy hívjuk, hogy a **lapon belüli eltolás értéke 6**.

A kérdés az, hogy hol van az operatív memóriában az 1. számú logikai lap? - ezt a laptábla mutatja meg.

Fizikai címszámítás

A processzor által kiadott *logikai címet formálisan két részre osztjuk*.

A cím első része adja meg a *lapszámot*, míg a második része a *lapon belüli eltolást*.



Forrás: Knapp G: OS

Fizikai címszámítás

Tegyük fel, hogy a *logikai cím* 16 bites, azaz $2^{16} = 65536$ sort címezhetünk meg.

Legyen egy lap mérete $2^{11} = 2048$ sor.

Ebből kiszámítható, a *lapok száma*, azaz: $2^{16} / 2^{11} = 2^5 = 32$ db lapunk van.

Tehát, a *lapok kiválasztására* 5 bit, míg a *lapon belüli sor kiválasztására* (eltolás) 11 bit kell.

Fizikai címszámítás – mintapélda

Adott egy *logikai cím*. Határozzuk meg a fizikai címet?

- logikai cím: 00010|10110101110 - ebből a
- lapcím: 00010, eltolás: 10110101110.

Látható, hogy a laptábla 00010 (=2) sorában az 10111 (=23) számérték található, azaz a keresett

memóriahely fizikai címe: 10111|10110101110.

Bázisregiszter – mi ez, hol van?

A programok betöltésekor *a fordítóprogram* nem *közvetlen generálja a fizikai címeket*, hanem a **program elejéhez képesti logikai *címeket***.

Ehhez azt kell tudni, hogy hol *kezdődik a program a memóriában*.

Ha ezt tudjuk, akkor ehhez a *kezdőcímhez* hozzá kell adni a programban szereplő *logikai* címet és megkapjuk a *keresett memóriarekesz fizikai címét* – ennek a támogatására fejlesztették ki a - **bázisregisztert**.

Bázisregiszter

A program kezdő- vagy báziscímét a regiszter tartalmazza.

Kérdés: ki határozza meg a program kezdőcímét?

Válasz: az **OS** - tudja, hogy éppen meddig ér és hol kezdődik *a szabad hely a memóriában.*

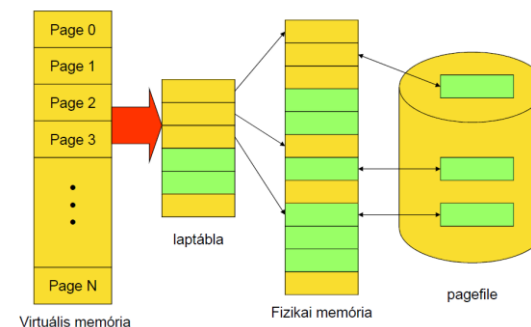
A program az első szabad címtől kezdve töltődik be, ezt a kezdőcím értéket, pedig a program betöltésekor beírja a bázisregiszterbe.”

Lapozásos virtuális memória kezelés

Az alapja a *lapozás* - a *folytonos virtuális címteret* egy tábla képzi le (page table, laptábla)

- Nem direkt módon *fizikai memóriára* történik a leképzés, hanem...részben *fizikai memóriára*, részben a *permanens táron* (HDD, Flash tár) kialakított *speciális területre*.

Pagefile (Windows), vagy *Swap file* (UNIX/Linux esetén swapping).



Forrás: OPRE jegyzet, BME

Lapozásos virtuális memória kezelés

Allokálás

A processz a *memória területet* (a virtuális memóriát) azonos méretű blokkokra, ún. **lapokra** osztjuk.

A fizikai memória ilyen méretű **lapkereteket** tartalmaz.

(Tipikus lapméret 2-4 kbyte, 64kbyte, 4M,

Linux x86: 4K (normal) / 4M))

A lapokat bármelyik *szabad lapkeretbe elhelyezhetjük* vagy akár a *háttértárolón kialakított területen* is.

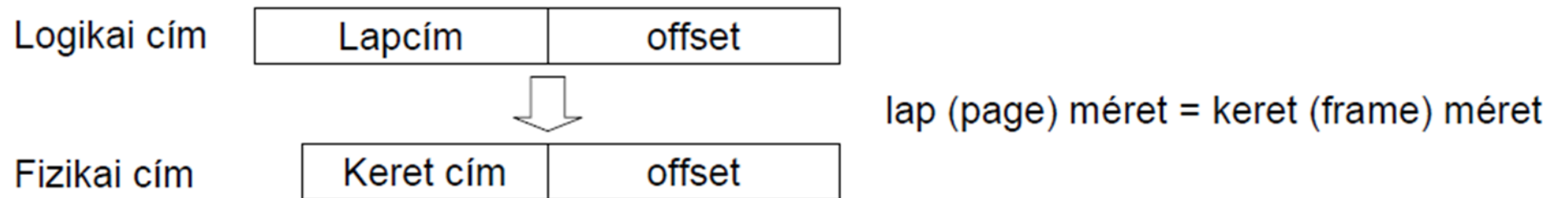
Lapozásos virtuális memória kezelés - Lapszervezés

Címképzés:

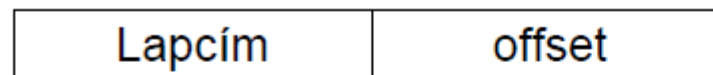
$$V \rightarrow R$$

V: a processz által használt *virtuális cím (Virtual)*

R: a *fizikai cím (Real)*



Lapozás: Logikai cím



Fizikai cím



lap (page) méret = keret (frame) méret

*A V két adattal megadható: **(p, o)** ahol*

- **p**: a lap címe (page)
- **o**: a lapon belüli eltolás (offset)

*Az R szintén két részre osztható: **(f, o)** ahol*

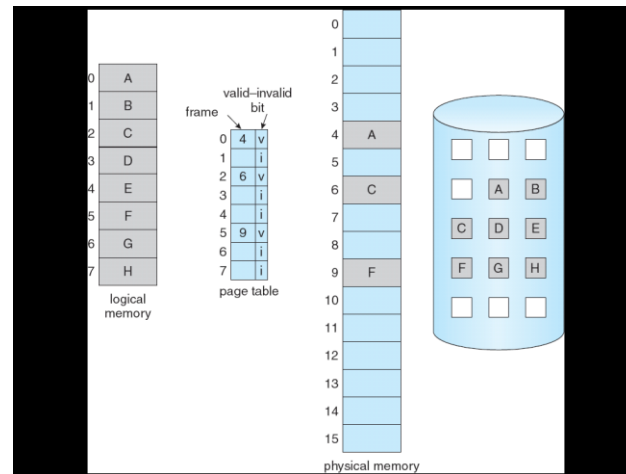
- **f**: a lapkeret címe (frame)
- **o**: ugyanaz a lapon belüli eltolás (offset)

Lapozásos virtuális memória kezelés

Címképzés:

Az OS minden processzhez létrehoz egy laptáblát.

*A laptábla kezdőcíme a processz kontextusából elérhető
(regiszterbe betöltve).*



Lapozásos virtuális memória kezelés

A laptáblának annyi bejegyzése van, mint ahány lapra felosztható a processz memória területe.

Egy **bejegyzés** tartalmazza, hogy:

- a *lap* a memóriában *van-e vagy sem* (valid/invalid),
- a helyét a *memóriában* (lapkeret cím: *f*),
- a helyét a *háttértáron*,
- egyéb adatokat, pl. hozzáférési tulajdonságok

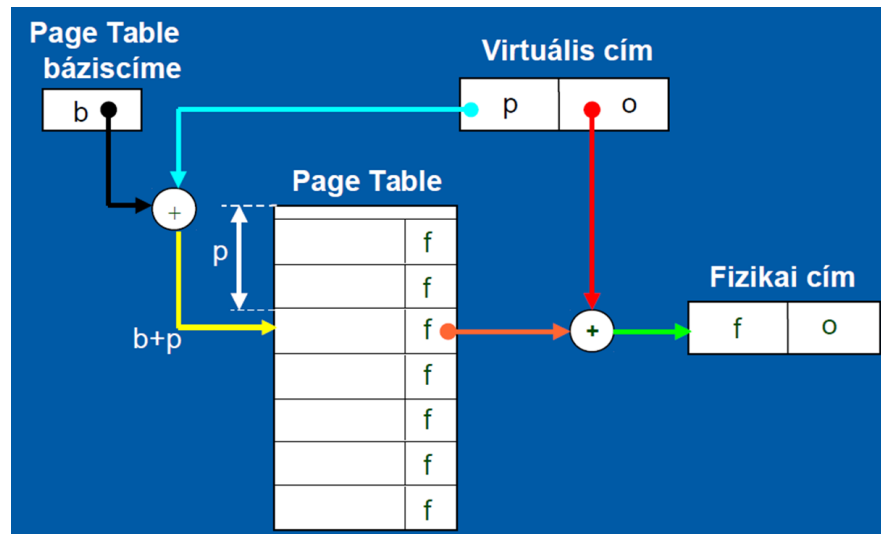
Lapozásos virtuális memória kezelés

Címképzés:

1. A V címet átalakítja (p, o) alakúvá.
2. A laptábla *kezdőcíméből* (b) és a *lapcíméből* (p) meghatározza a laptábla p -hez tartozó bejegyzését.
3. Ha a *bejegyzésben* azt látja, hogy a lap nincs a memóriában, akkor ezt nevezik **laphibának**.

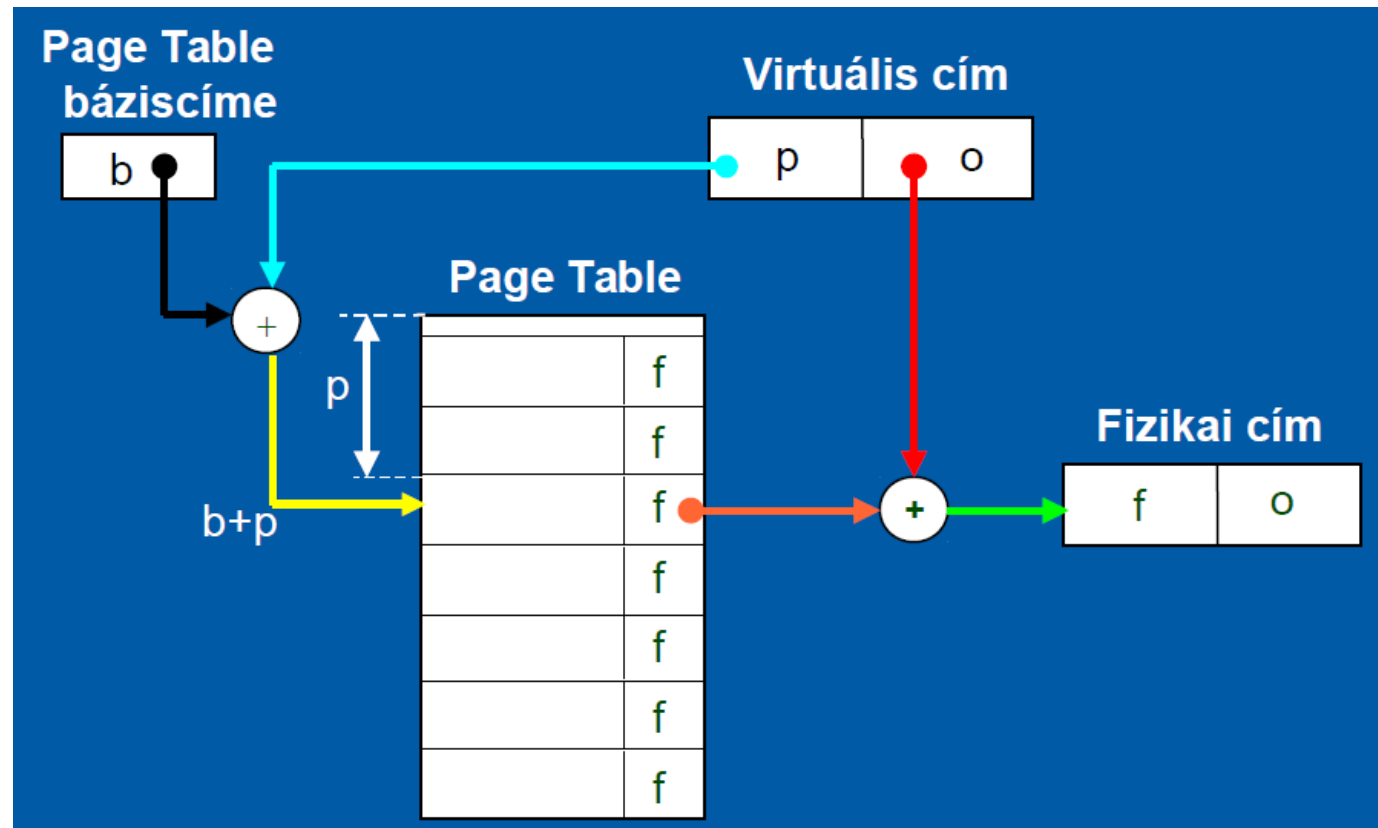
Lapozásos virtuális memória kezelés

4. Ha a memóriában van, akkor a *bejegyzésből* megkapja a lap fizikai címet (f).
5. Az f-hez hozzáadja a *lapon belüli eltolást* (o) és megkapja R-t.



Lapozásos virtuális memória kezelés

Lapozásos virtuális memória kezelés



Lapozásos virtuális memória kezelés - laphiba

Címképzéskor lehet **laphiba**:

A hivatkozott lap nincs betöltve lapkeretbe.

(kivételes esemény – exception!)

1. Laphiba esetén a *processz blokkolódik*.

Az *esemény lekezelője* az OS ezzel foglalkozó része a *pager*.

Lapozásos virtuális memória kezelés - laphiba

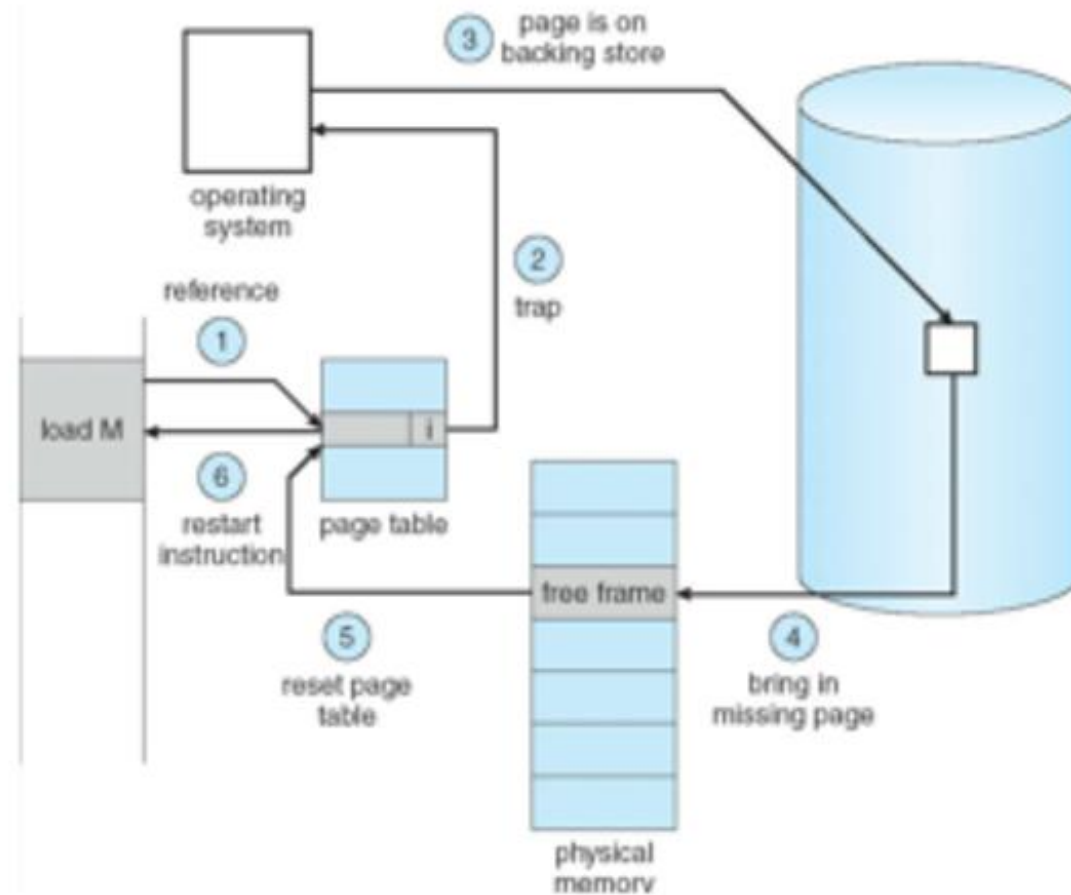
2. A *pager* betölti a kívánt lapot.

Ha van a processznek üres lapkerete, akkor oda, ha nincs akkor helyet csinál - lásd *kilapozási algoritmusok*.

3. A processz visszakapja a futási jogot és ha futásra kerül, akkor a laphibát okozó utasítás ismételt végrehajtásával folytatja.

Lapozásos virtuális memória kezelés - laphiba

1. Érvénytelen laphivatkozás.
2. Megszakítás.
3. Lap háttértárról előkeresése.
4. Lap üres keretbe betöltése.
5. Laptábla aktualizálása.
6. Utasítás újraindítása.



Lapozásos virtuális memória kezelés - laphiba

A processz által látott *folytonos címteret* meg kell különböztetni a *valós címtértől*.

- A kettő közti konverziót az OS *laptábla* segítségével végzi el.
- *Laptábla feladata: egy lap* csak akkor kerüljön be a *fizikai memóriába*, amikor *ténylegesen egy processz hivatkozik* rá.
- *Megvalósítás:* a laptábla minden bejegyzéséhez tartozik egy *valid/invalid bit*.
- Ez azt tárolja, hogy az *adott bejegyzéshez* tartozó *memóriatartalom be van-e töltve* a fizikai memóriába.

Lapozásos virtuális memória kezelés - laphiba

- Ha egy olyan lapra hivatkozunk, amely *invalid*, tehát még nincs a memóriában, akkor egy *laphiba generálódik* az MMU által.
- Ez nem hibás működés, hanem csak azt jelenti, hogy azt a lapot be kell tölteni.
- *Megvalósítása: a hiba* hatására a processz működése megszakad.
- Az OS behozza a *fizikai memóriába* a hivatkozott lapot.
- Majd visszaadja a vezérlést a processznek.

Lapozásos virtuális memória kezelés - laphiba

A laphiba a lapozásos virtuális memória kezelés természetes velejárója.

A laphiba kezelési mechanizmus teszi lehetővé, hogy nem szükséges a processz teljes memória tartalmát ténylegesen a memóriában elhelyezni!

Lapozásos virtuális memória kezelés - laphiba

DE! Az *instrukció végrehajtásának ideje*, ha laphiba is előfordul közben több **ezerszerese** is lehet, mint ha nem fordul elő laphiba.

Fontos, tehát a *laphibák számának csökkentése*.

Ezenkívül fontos a *laphiba lekezelés idejének csökkentése*.

Lapozásos virtuális memória kezelés - védelem

Védelem

Külön védelemre nincs szükség, hiszen a *leképzési mechanizmus biztosítja*, hogy a *processz csak saját területeihez tud hozzáférni*.

Lapozásos virtuális memória kezelés - SHM

SHM

Osztott memória kezelés un. közös lapokkal valósítható meg.

A processz laptáblájában egy bejegyzés jön létre a laphoz, melyet az OS külön nyilvántartja.

Egy másik processz csatlakozásakor az ő laptáblájába is kerül egy *bejegyzés* ugyanarra a lapra.

Lapozásos virtuális memória kezelés - előnyök

Előnyök:

- Nincs *külső töredezettség*.
- *Nagyon kicsi belső töredezettség* (csak a processz utolsó lapja lehet kisebb méretű, mint a lapkeret).
- Könnyű elhelyezés. Bármelyik lap, bármelyik lapkeretbe.
- Nagyon nagy memória igényű processzek is kezelhetők (a fizikai memóriánál nagyobb is).
- Nincs szükség sem *overlay-re* sem *swapping-re*.

Lapozásos virtuális memória kezelés

Hátrányok:

- *A címleképzés ideje jelentősen megnőtt.*
- *A nyilvántartás mérete megnőtt.*
- *A nyilvántartás kezelési ideje megnőtt.*
- *A laphibák számának és kezelési idejének csökkentése érdekében újabb OS funkciókra van szükség.*

Keresés a laptáblában

A keresés a laptáblában lassú:

- 2x annyi idő (laptábla indexelés és olvasás majd adatelérés).
- *Asszociatív lekérdezés* (Translation look-aside buffer, TLB) kombinálva a laptábla használatával.

A *cél* tehát, az hogy csökkentse a *memória-hozzáférés idejét* olyankor is, amikor nem volt laphiba!

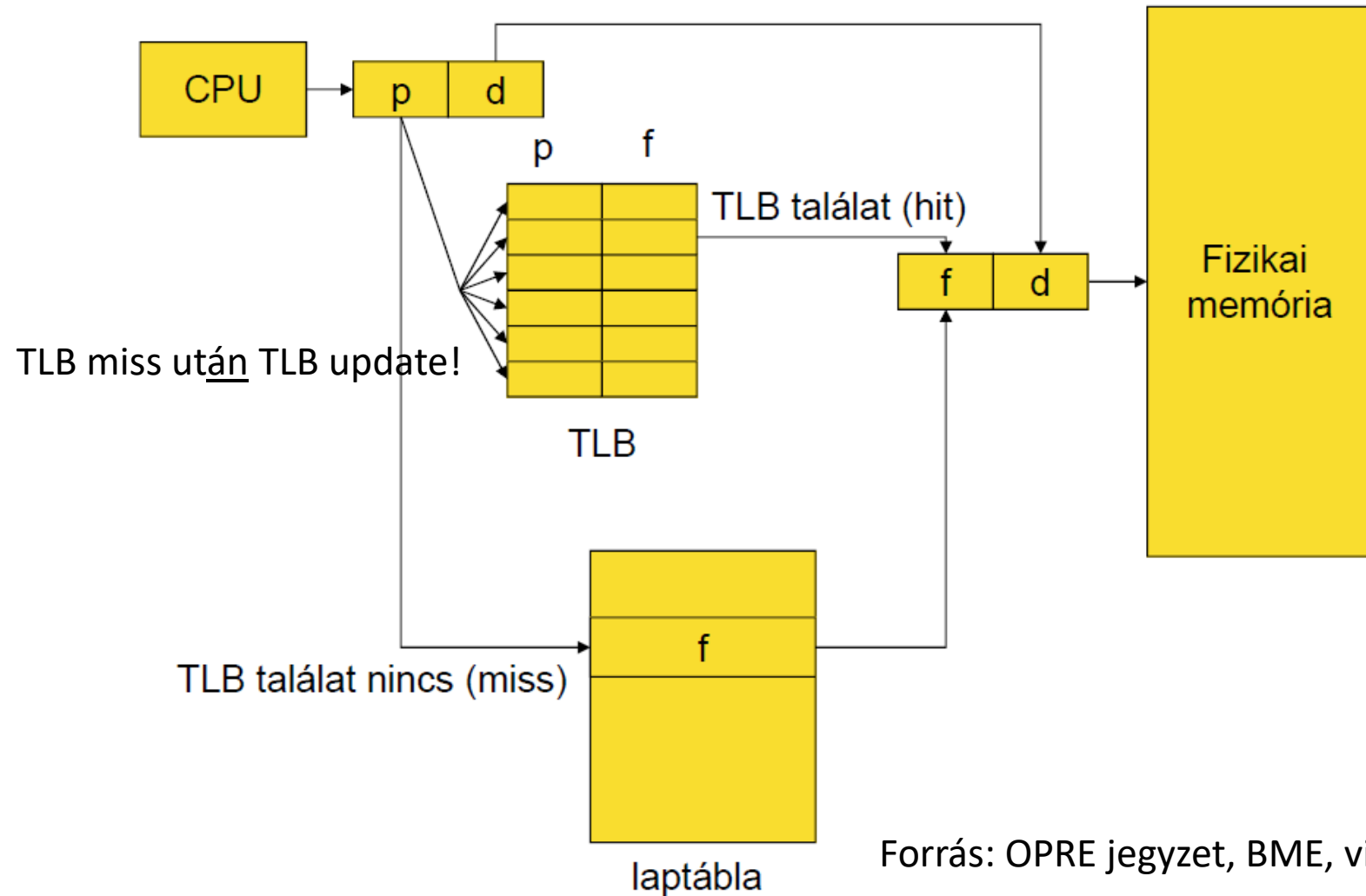
Keresés a laptáblában - Asszociatív leképzés

Asszociatív tárok használata: párhuzamosan egy asszociatív tárból is keres. Hogyan?

Translation lookaside buffer (TLB): ez egy gyorsítótár.

Feladata: címfordítás gyorsítása, azaz mely *virtuális cím*, mely *fizikai címhez* van rendelve.

Keresés a laptáblában - Asszociatív leképezés



Lapozásos virtuális memória kezelés – Lapozási stratégiák

Lapozási stratégiák lehetnek:

- *Igény szerinti* belapozás (demand paging): csak akkor lapozzuk be ha *hivatkozás történt* rá, csak a hivatkozott lap töltődik be a memóriába. Egyszerű megvalósítás, de lassú.
- *Előérző (előrettekintő) belapozás*: az OS előre megjósolja milyen lapra lesz szükség, a **lokalitás elveket** figyelembe véve (időbeli, térbeli)

Lapozásos virtuális memória kezelés - *lokalitás*

Időbeli: a hivatkozás címet a közeljövőben újra használni fogják.

Térbeli: a hivatkozás egymás melletti címekre történik:

- Sorosan végrehajtott kódrészletek
- Tömbkezelés
- Listakezelés

Lapozásos virtuális memória kezelés – Lapcsere stratégiák

A virtuális tárkezelési módszer esetén elkerülhetetlen az a pillanat, hogy a fizikai memória megtelik, azaz nincs üres hely benne, ahova behozható lenne az új memórialap.

Ilyenkor lépnek életbe a lapcserélési stratégiák, amelyek meghatározzák, hogy melyik lap helyére töltődjön, majd be az új memóriatartalom.

Lapozásos virtuális memória kezelés – Lapcsere stratégiák

A pager feladata, hogy ha a processznek nincs már szabad lapkerete, akkor egy lapot ki kell lapozzon a memóriából.

Lokális kilapozási stratégiák: csak a processz saját lapkeret készletéből lapozhat ki.

Globális kilapozási stratégiák: más processzek lapkeret készletéből is kilapozhat.

Lapozásos virtuális memória kezelés – Lapcsere stratégiák

0. OPT – optimális algoritmus.

1. Legrégebbi lap algoritmus (*First-In-First-Out*) .

2. Újabb esély algoritmus (*Second Chance*).

3. Legrégebben nem használt algoritmus (*Least Recently Used*).

4. Legkevésbé használt algoritmus (*Least Frequently Used*).

5. Utóbbi időben nem használt algoritmus (*Not Recently Used*).

Lapozásos virtuális memória kezelés – Lapcsere stratégiák

A laphibák *két csoportra oszthatók*.

- Az első *három/négy* (memória kerettől függ) elkerülhetetlen, hiszen valamikor fel kell *tölteni az üres kereteket*,
- A többi viszont *az algoritmus jellemzője*.

Lapozásos virtuális memória kezelés – Lapcsere stratégiák

Igényelt lapokat a kétféle módon helyezhetjük el a memóriában:

- lapok hivatkozása (pozíciója) a memóriában *ténylegesen elfoglalt fizikai helyet jelöli,*
- *logikai sorrendet* hivatott megjeleníteni.

Lapozásos virtuális memória kezelés

Kilapozási stratégiák - OPT

0. OPT - optimális

Ez az *algorithmus optimális*, tehát a „legjobb”.

Előny: ez adja a *minimális laphiba* számot.

Hátrány: a gyakorlatban megvalósíthatatlan, mert nem tudhatjuk előre a lap *hivatkozások sorrendjét*.

Az új lapot mindig annak a helyére hozzuk be, amelyre a *legkésőbb fogunk (újra)hivatkozni* – legkésőbb lesz szükség.

Lapozásos virtuális memória kezelés

Kilapozási stratégiák - FIFO

1. FIFO - Előbb be, előbb ki: fontos a belapozási sorrend.

- A behozott lapokat a *behozás sorrendje szerint sorba rendezi*.
- Laphiba esetén a *FIFO elején álló lapot cseréli* le.
- Minden frissen *behozott lap sorszáma* bekerül a *FIFO sor végére*.
- A sorban már bent levő lapsorszámok eggyel előre lépnek és a sor másik végén "kipotyog" a legrégebbi lap sorszáma.

Lapozásos virtuális memória kezelés

Kilapozási stratégiák – második esélyes FIFO

2. Második esélyes FIFO - a FIFO továbbfejlesztése

Minden laphoz egy *hivatkozás bit (used bit) tárolódik* - volt-e a közelmúltban használva.

- Ha hivatkoznak egy lapra 1-re áll.
- Kilapozáskor a *sor elején levő* lapnak (legrégebbi lapnak)
 - ha 0 a hivatkozás bitje - akkor kilapozódik,
 - ha 1, akkor 0 lesz és a sor végére rakja.

Lapozásos virtuális memória kezelés

Kilapozási stratégiák – LRU

3. LRU - Least Recently Used: a legrégebben nem használt.

Az algoritmus egy aránylag bonyolult megoldás, de jól megközelíti az optimális megoldást.

Ha egy lapra hivatkoznak sor végére kerül. Viszonylag költséges algoritmus.

A lapcserét az alapján választja ki, hogy melyik volt legutoljára használva.

Lapozásos virtuális memória kezelés

Kilapozási stratégiák – LRU

- *Számlálós megvalósítás:* minden laphoz egy időbélyeget ment el a rendszer, amely azt tárolja, hogy *mikor volt utoljára használva*.
- *Láncolt listás megvalósítás:* A *legutoljára használt lap* mindig egy *láncolt lista végére kerül* és így a lap mindig *kiválasztható a lista elejéről*.

Lapozásos virtuális memória kezelés

Kilapozási stratégiák – LFU

4. LFU (*Least Frequently Used*): a legkevésbé használt.

- Az algoritmus megszámolja, minden lap esetében, hogy hány hivatkozás történt rá, így *amelyikre a legkevesebb hivatkozás történt az lesz lecserélve.*
- Nyilvántartani kell a hivatkozások számát → költséges.

Lapozásos virtuális memória kezelés

Kilapozási stratégiák – NRU

5. NRU (*Not Recently Used*) – nem túl gyakran használt.

A kis költség a cél. Többféle megvalósítás. Pl.: *módosítás bites*

Minden laphoz egy **R** (hivatkozott) és egy **M** bit (módosított).

- *M bit* jelzi, hogy *módosítottak-e* a lap tartalmat.
- *R bit* *hivatkozáskor* 1-re, (bizonyos időnként 0-ra,) jelzi, hogy hivatkoztak-e a lapra.

Lapozásos virtuális memória kezelés

Kilapozási stratégiák – NRU

A két bit értéke alapján *egy prioritási sorrend* határozható meg:

- 0. szint: $R = 0$ és $M = 0$ (*legalacsonyabb szint*)
- 1. szint: $R = 0$ és $M = 1$
- 2. szint: $R = 1$ és $M = 0$
- 3. szint: $R = 1$ és $M = 1$ (*legmagasabb szint*)

Az algoritmus mindig a *legalacsonyabb prioritású szintről* választ egy lapot a cseréhez.

Memória kezelés - Szegmensek

Memória kezelés - Szegmensek

Memória kezelés - Szegmensek

*A logikai címtartomány **szegmensekre** van osztva.*

- *A programozó egy szegmenst és azon belül egy szegmens ofszetet ad meg.*

Memória kezelés - Szegmensek

A virtuális címtartomány nem egydimenziós.

Külön címtartományok

*a kódnak, az adat szekcióknak, a veremnek,
az osztott kódnak, kernel régióknak, stb.*

Ezek a szegmensek.

Ez fordító támogatást igényel.



Folyamat a memóriában

Memória kezelés – Szegmensek - allokálás

A virtuális memória *szegmensekre* van felosztva.

A szegmenseket *memóriában* vagy a *háttértáron* kell elhelyezni.

Tehát, *nem egyforma méretűek a blokkok*.

Elhelyezés problémái: hasonlóak a *változó partíciós memória* kezelésnél tanultaknál, (mint ott egy partíció).

Memória kezelés - Szegmensek - allokálás

Elhelyezés (allokálás):

- Elhelyezési algoritmusok: first fit / next fit / best fit / worst fit
- *Nyilvántartás* szükséges a *szabad területekről*.

Memória kezelés – Szegmensek - címképzés

Címképzés:

Virtuális cím: (s, d)

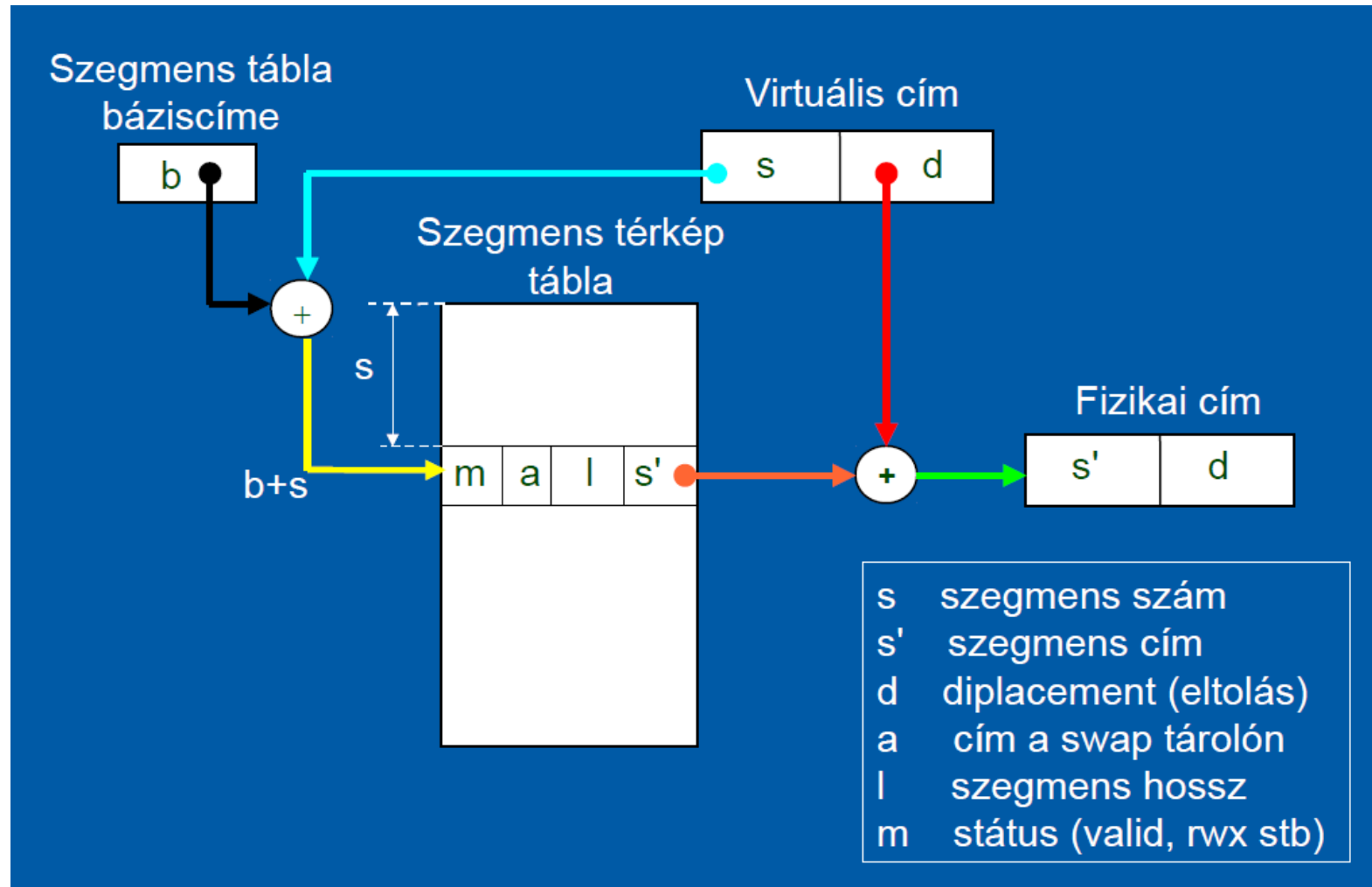
Nyilvántartás kell a processzhez tartozó szegmensek elhelyezkedéséről - ez a szegmenstábla.

Memória kezelés - Szegmensek - címképzés

Ebben *egy bejegyzés*:

- szegmens *memóriában-e vagy sem*.
- szegmens *elhelyezkedése a memóriában*.
- szegmens *elhelyezkedése a háttértáron*.
- *szegmens hossz*.
- egyéb adatok (pl. r, rw).

Szegmensenkénti címleképzés



Memória kezelés – Szegmensek - Allokálás

Szegmenshiba: ugyanaz, mint a laphiba.

Szegmenshiba kezelése ugyanúgy, mint lapozásos memória kezelésnél.

Védelem

Szegmensen *belüli eltolás* és *szegmenshossz összevetése*.

Belső védelem: szegmensekhez jogosultságok hozzárendelésével (pl. kódszegmensek csak olvashatóak)

Memória kezelés - szegmentálásos virtuális memória

SHM

Osztott szegmenseket használ, a lapozásoshoz hasonlóan.

Előnyösebb: mint osztott lapok, mert logikailag összetartozó adatok vannak egy szegmensben.

Memória kezelés - szegmentálásos virtuális memória

Előnyök

- Kevesebb *a szegmenshiba*, hiszen a szegmensekre osztás igazodik a *programok logikai szerkezetéhez* (lokalitás elv)!
- Kényelmesebb *osztott memória használat*.

Memória kezelés - szegmentálásos virtuális memória

Hátrányok

- Sokkal több munkája van az operációs rendszernek, mint a lapozásosnál: *elhelyezési stratégia, bonyolultabb nyilvántartás* etc.
- Van *külső elaprózódás*. A helykihasználás kisebb, mint a lapozásosnál.

Memória kezelés - Szegmentálós és lapozós virtuális memória

Szegmentálós és lapozós virtuális memória

A két módszer ötvözése:

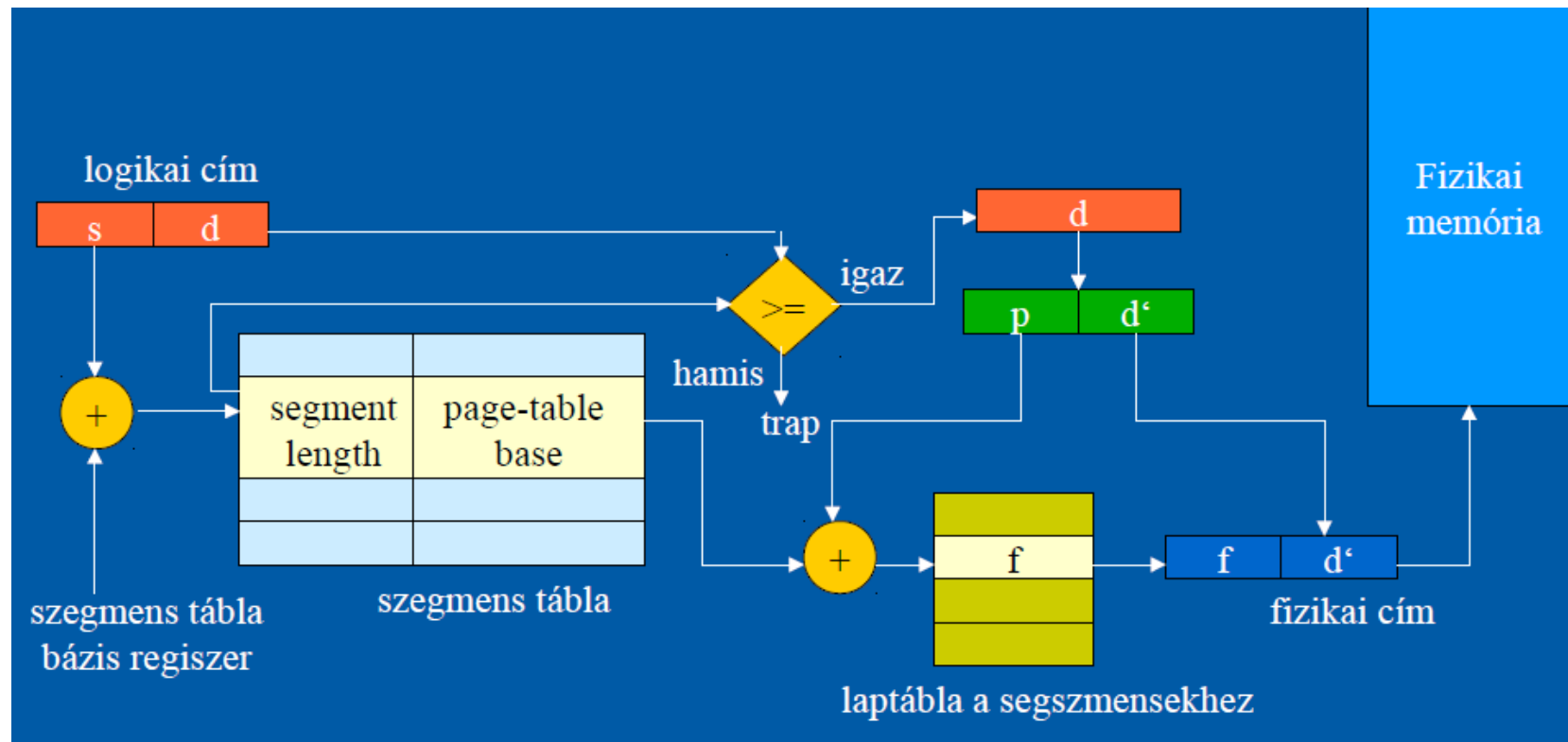
- A processz memória területe *szegmensekre* van osztva.
- A szegmenseket kell elhelyezni vagy a *memóriában* vagy a *háttértáron*.
- Az elhelyezés *szegmenstáblában* (szegmentálás).

Memória kezelés- Szegmentálásos és lapozásos virtuális memória

- *Előnyei:* kevesebb szegmenshiba, osztott szegmensek.
- A szegmensek memóriában *elhelyezésekor lapokra bontás.*
Lapozás előnyei: nincs elhelyezési stratégia, bonyolult a (üres) nyilvántartás, külső elaprózódás.

Memória kezelés - Szegmentálós és lapozós virtuális memória

Minden memóriában levő szegmenshez tartozik egy laptábla.
Bonyolultabb címképzés:



WinNT memória kezelése

Virtuális, lapozós

- *Két szintes laptáblát + VAD-ot (V.A.S) használ a lapok nyilvántartásához (Virtual Address Descriptor).*
- *Lapkeret adatbázist használ a szabad lapkeretek kereséséhez.*
- *Munkakészletet használ, igény szerinti belapozós.*
- *Önálló taszk a kilapozó.*

WinNT memória kezelése

- 32 bites címezés: 2^{32} lineáris címtartomány (4GB)
- A felső 2GB *rendszer címtartomány*, kernel módban látható (Pentium 1GB).
- Az alsó 2GB *taszkonként egyedi címtartomány* (Pentium 3GB).

WinNT memória kezelése - VAD

Virtual Address Descriptor

- A VAD a *lefoglalt címtartományok leíróit* (kezdő-végcím, további adat) *tartalmazza, bináris fa struktúrába rendezve* (gyors keresés).

Szerepe:

- Címleképzésnél is: először a VAD struktúrát nézik: lefoglalt címtérületre történt-e a hivatkozás?

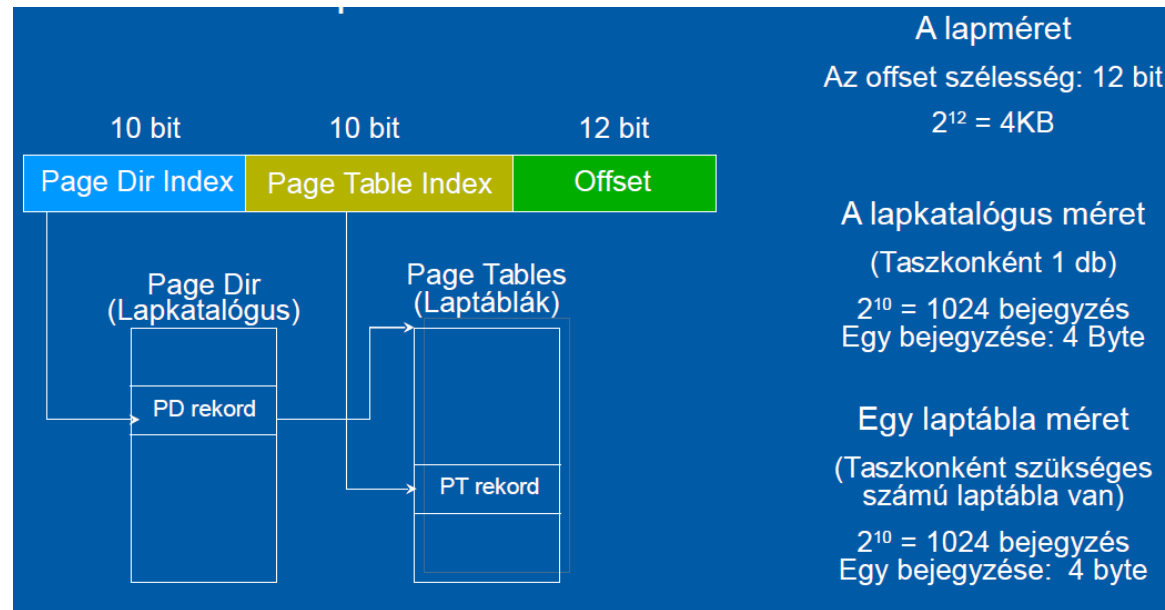
WinNT memória kezelése

- *A memória allokálásnál:* új VAD leírót kell felvenni + Vmet hozzárendelni az új címtartományhoz:
- Először csak VAD leírót veszünk fel (alloc), amikor tényleg használni akarjuk az új mem-et (committing), elkészítjük a laptábla bejegyzéseket stb.

WinNT memória kezelése - Címleképzés x86-on

Címleképzés x86-on

- TLB-t is használ (asszociatív tár)
- Kétszintű laptábla



WinNT memória kezelése

Laptábla bejegyzés tartalma

- *Státusz bitek* (érvényes, nem érvényes, prototípus laptábla index jelzésére)
- *Védelmi bitek* (read only lap, r/w lap, exec only lap, figyelt lap (elérése kivételt generál),
- *Tiltott lap* (elérése kivételt generál),
- *Zárolt lap* (nem kilapozható),
- *Módosításkor másolt* (copy on write) lap

WinNT memória kezelése – Laptábla bejegyzés tartalma

- *Lapkeret* cím.
- Másodlagos tároló blokk cím.
- Prototípus laptábla index. (osztott memóriához).

WinNT memória kezelése – Lapkeret adatbázis

Annyi bejegyzése van, ahány lapkeret van. Tartalma:

- *Érvényes* (valid) keret: használatban van.
- *Szabad* (free) keret.
- *Nullázott* (zeroed). Szabad és 0-kkal felülírt keretek.
- *Készenléti* (standby) keret: már felszabadított, de még benne az érintetlenül hagyott adatok. Visszakérhetők belapozás nélkül, csak érvényesre kell állítani őket.

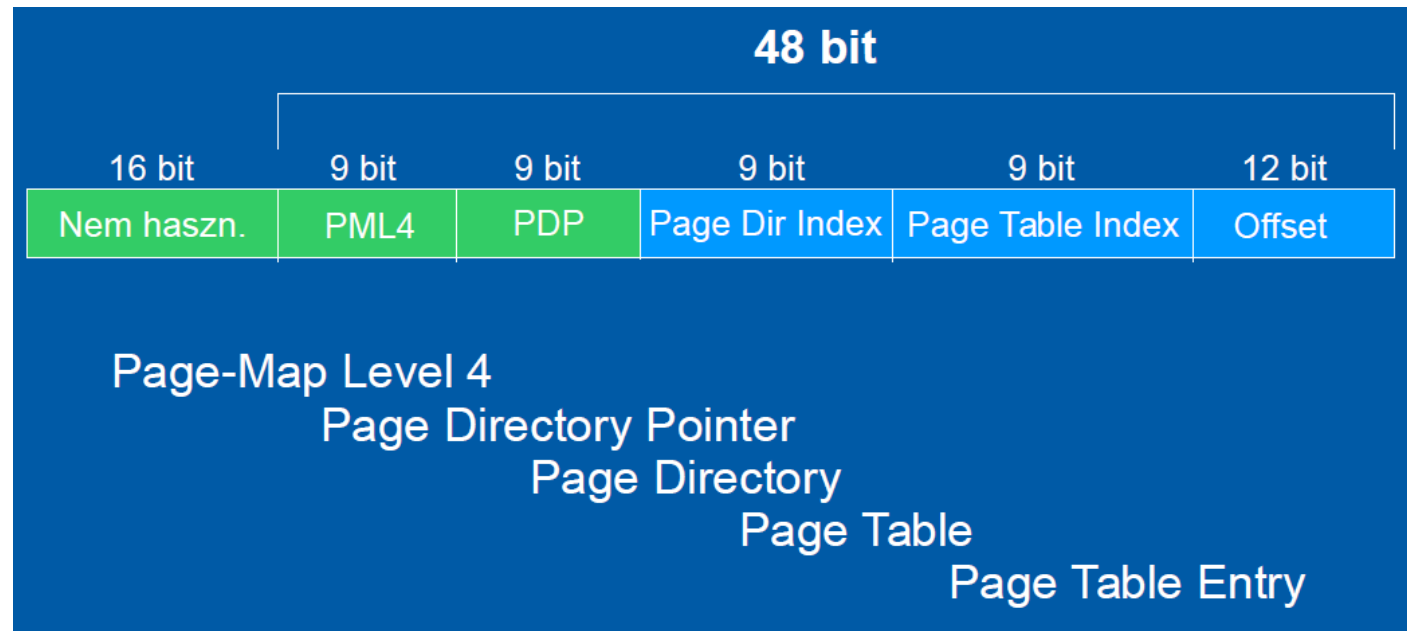
WinNT memória kezelése – Lapkeret adatbázis

- *Módosított* (modified) *keret* (kilapozáskor ki kell írni).
- *Hibás* (bad) *keret*: megbízhatatlanul működik.
- Visszamutató a taszk laptábla bejegyzésére.

64-bit memória kezelés

Címleképzés (64-bit x86 – x64 – x86_64 – EM64T...)

- Négy szintű laptábla



Linux memória kezelése

- Szegmentáció nincs (de ettől még a szegmens regisztereknek a CPU-ban vannak értékeik!)
- *Lapozós virtuális memória*
 - 4 KByte méret (4 MByte is támogatott),
 - 64-bit címtartomány,
 - 3 szintű laptábla.

Linux memória kezelése

- x86-on:
 - itt csak 2 szintet használ
 - (64-biten pedig 4 szintet (48-bit))
 - Lapozás HW támogatott, TLB is.
- Igény szerinti belapozás.

Linux memória kezelése

- Laptáblák
 - Global Directory
 - Middle Directory
 - Page Table
 - Offset

User mód/Kernel mód memória rész.

Linux memória kezelése

Processz szemszögéből:

- Processz logikai címekkel dolgozik.
- Címtérület – az egész ami használható.
- Ez minden processznek külön.
- Memória régiók – az allokálás „darabjai”
 - Mérete a lapméret többszöröse
 - fork, exec, malloc (brk), mmap, shmat

Linux memória kezelése

- Memória deszkriptor (processz kontextusban)
 - Mindenféle adatok...
 - Konkrét memória részek egy láncolt listán
 - Darabka leírója + next pointer

Linux memória kezelése

- Processz szemszögéből
 - Foglalásnál nem feltétlen rendelődik egyből fizikai memória is processzhez.
- Laphiba esetén:
 - Elérni kívánt cím összehasonlítása a címterülettel.
 - Ha nincs benne → hibás elérés (SEGV).
 - Tényleges fizikai memória foglalás.
 - Swap!

Linux memória kezelése

- CoW – Copy on Write
 - Nem készül tényleges másolat, csak hivatkozás.
 - De, amint írás történik, készül egy másolat, és oda történik az írás.
 - pl. fork()-nal”[1][2][3][4][5][6][7]

Irodalom

[1] Vadász Dénes: Operációs rendszerek, ME, 2007.

<http://www.iit.uni-miskolc.hu/~vadasz/GEIAL302B/2007-Ea>

[2] Operációs rendszerek

<https://dcs.uni-pannon.hu/oktatas/tantargyak/operacios-rendszerek>

[3] Tannenbaum: Modern Operating Systems, Prentice Hall, 1992.

[4] Kóczy, Kondorossi szerk: Operációs rendszerek mérnöki megközelítésben, Panem, 2000.

Irodalom

[5] Operációs rendszerek

<https://vik.wiki.hu>

[6] Heizlerné Bakonyi Viktória, Horváth László, Illés Zoltán, Nagy Tibor, Lutár Patrícia: Számítógépes alapismeretek II. ELTE 2012.

https://regi.tankonyvtar.hu/hu/tartalom/tamop412A/2010-0011_szamalap2/adatok.html

[7] Knapp Gábor: Operációs rendszerek, 2000.